

An Experience Report on a Specification-Driven, AI-Assisted Software Construction Project

Status: complete and verified against the course template. Sections 1 to 7 and the appendix are written from the repository, and every count and measurement in them is recomputed from the repository by `scripts/check_report.py`, which runs in `make check-strict`. One item is outstanding and is marked as such where it belongs rather than implied complete: the user acceptance testing in `docs/vv/acceptance.md`, which needs a person who is not the author and is the reason NFR-007 and SC-005 are the two requirements the traceability matrix still shows as unmet.

Team Details (Team A)

Student Name	Student ID	Role in the project	Contribution (%)
Theo Wilenius	2202234	All four roles, rotated per phase: specifier, prompt engineer, verifier, auditor	100

The project is done solo, against the course recommendation of teams of three. Section 6.2 says how the four roles were separated in time, and how independent agent review in a fresh context, self-review against `CONTRIBUTING.md`, and planned external acceptance testing substitute for a second and third member.

Executive Summary

The system built is **ScreenDelta**, a deterministic frame delta and element identity engine in Go. It reads consecutive 1920 by 1080 frames, reports which rectangular areas changed, assigns each area an element identity that stays stable while that element is tracked, and emits one self-contained JSON document per frame. It is deliberately one stage of a larger pipeline: it does not classify, recognise text or drive a pointer, and it carries an explicit uncertainty marker for identities it has evidence for but will not claim.

Specification was formal and versioned: a project constitution, a feature specification with 16 functional requirements, 11 non-functional requirements and 6 success criteria, an implementation plan with research decisions, interface contracts including a JSON Schema, a task list of 53 tasks, and a requirement traceability matrix checked by a script that runs in CI.

The work used **GitHub Spec Kit v1.0.12** as the SDD tool kit, driven command by command from a DeepSeek backed agent harness, with the same repository reachable from three surfaces: GitHub Copilot prompts, opencode commands and dsh skills. Independent review ran in a separate agent context, briefed to falsify rather than to read, and produced six rounds and 46 findings.

Verification is measurement rather than assertion, and every measurement was first defeated deliberately: F1 1.0000 over 5,134 generated frame pairs with zero false removals, p95 latency 9.65 ms per 1080p frame pair on one CPU core against a 12 ms target, 79.2 frame pairs per second, 24.0 MiB peak heap over 10,000 frames against a 128 MiB ceiling, byte-identical output across thread counts and collector settings, and 29 of 33 requirements verified with a committed test and recorded output. Thirty-five defects were found and recorded; 34 are fixed and one is documented as inherent to a pixel-only stage.

1. Introduction and Paradigm Shift (3 points)

1.1 Project Overview and Objectives

ScreenDelta serves a planner: a program that decides what to click or type next by looking at a screen. Modern models are good at deciding and slow at looking, because finding out what changed between two frames of a legacy interface is a solved problem that nobody has packaged with an honest contract. The engine therefore does one thing well and chains: frames in, a versioned delta document out.

The objective was set by the course constraint that the component must be small enough to finish properly and useful enough to sit in a pipeline. Eleven candidate systems were scored on stakeholder access, requirement richness, verification strength, demonstration value, blocker risk and how much leverage an agent team gives, and the scores are in docs/ideas.md. The selection decision and the alternatives rejected are in docs/adr/0002-frame-delta-engine.md; the choice of Go over Rust, C++ and Python is docs/adr/0003-go-implementation.md, decided on build and test speed across Linux and Windows, a standard library that covers image decoding without a dependency, and a cross-compilation story that makes the Windows requirement cheap rather than expensive.

The engine deliberately excludes what the pipeline can do better: no machine learning, no OCR, no element classification, no capture implementation beyond reading files and standard input. Each exclusion is a decision in specs/001-frame-delta-engine/research.md with the reason recorded.

1.2 The Vibe Coding

Vibe coding, in the sense used here, is prompting an agent with an outcome and accepting what it produces without a specification to check it against. It was used for exactly three things, each deliberately bounded:

- **exploration before the specification existed**, where an agent was asked what a chainable GUI change detector could usefully promise, which produced the candidate list in docs/ideas.md and was thrown away afterwards except for the scores;

- **repository and process scaffolding**, where the first session produced the Spec Kit layout, the process documents and the traceability script from a description of what the course grades;
- **wording**, where error messages, documentation and commit messages were drafted by an agent and corrected by hand.

What was kept from the unstructured phase is small and auditable: the idea scores, the process layer and the wording. What was not kept is the code: no implementation line was written before the requirement it implements existed, which the repository enforces rather than promises, because `scripts/check_traceability.py` fails when a requirement has no task and a task has no test.

The cost of the unstructured phase was real and is recorded in `docs/prompt-log/0005-system-selection.md`: four candidate systems were explored in detail before the fifth was chosen, and the first two were rejected for reasons that a single question about the course grading would have surfaced immediately. The lesson, applied since, is that the exploration prompt has to name the constraint it is optimising for.

1.3 The SDD Methodology

Specification-Driven Development was chosen because the failure mode it prevents is the one this project would otherwise have: an agent produces plausible code quickly, nothing records what it was supposed to do, and the first real test becomes the specification. The chosen kit is GitHub Spec Kit v1.0.12, pinned by `uv tool install specify-cli --from git+https://github.com/github/spec-kit.git@v1.0.12`, for three reasons recorded in `docs/adr/0001-adopt-spec-kit.md`: it is the kit the course names, it is command-driven rather than document-driven so the same repository works from three agent surfaces, and its templates are overridable, which let the constitution and the report skeleton live in the repository rather than in the tool.

The specification scope is one feature, `specs/001-frame-delta-engine/`, with three user stories in priority order: a deterministic delta that a planner can act on, element identity across a stream, and a fingerprint that lets a plan be cached. The persistence model is flow-forward: the specification is updated deliberately when reality contradicts it, and the update is a commit of its own with the reason in the message, rather than the specification being regenerated from the code. Deviations that change the design rather than the wording become ADRs; deviations that change a requirement are spec commits.

Two diagrams accompany this section, because the template asks for both approaches. The unstructured loop is `docs/process/vibe-coding-workflow.svg`: describe an outcome, let the agent generate, try it, keep or discard, with no artifact accumulating between prompts, which is why its three failure modes are the reason this project did not write implementation code that way. The specification-driven workflow actually followed is `docs/process/sdd-workflow.svg`, and section 7.4 proposes the version of it worth reusing.

The development timeline so far is two working days of agent-assisted construction, 2026-09-26 and 2026-09-27, against a submission deadline of 2026-10-25. That is worth stating plainly, because the report is graded on what the process produced rather than on how long it took: more than 100 commits, 6,153 lines of Go and 5,899 lines of test, with the specification, plan and task list written before the code they describe. The feature workflow this repository commits to is drawn in docs/process/sdd-workflow.svg and described in section 7.4.

2. Requirements and System Specification (6 points)

2.1 Stakeholders

Stakeholder	What they need	Why selected
The pipeline author	One stage whose output can be planned against, with a schema that does not change under them	They are the immediate consumer, and the reason the engine is a stage rather than a tool
The course assessment	Traceable requirements, visible process, and a history that shows the work	The repository is a graded artefact, so the process is a deliverable
A future maintainer	The reason behind each non-obvious rule, available without reading the code	Recorded as decision records and research notes, because the interesting decisions are the ones that look arbitrary later
An agent working on the code later	Rules it can follow and checks it cannot pass without satisfying	AGENTS.md, CONTRIBUTING.md and the traceability check exist for this reader specifically
A screencast tool author	Frames in, deltas out, with no opinion about capture	Excluded from scope deliberately: capture stays outside, so the engine can be embedded

The stakeholder list drives the requirements rather than decorating them: FR-011 (regions of interest) and FR-012 (configuration) exist because the pipeline author needs to bound the work, FR-014 and NFR-010 exist because a schema that changes silently breaks the future maintainer, and FR-016 exists because a pipeline stage that writes files or opens sockets is not embeddable.

2.2 Functional Requirements

Functional requirements are written as testable statements with stable identifiers, each naming a behaviour rather than an implementation, and each traced to a task in docs/traceability.md. The full set is specs/001-frame-delta-engine/spec.md; what follows is the shape of it rather than a copy.

By user story and priority:

Story	Priority	Functional requirements	What it promises
US1: a deterministic delta	P1	FR-001 to FR-005, FR-010 to FR-016 (12)	Region detection with bounds, class, magnitude and area; a noise floor that is documented and configurable; one self-contained document per frame; explicit failure without a partial document
US2: element identity	P2	FR-006, FR-007 (2)	Identifiers stable across a stream and never reused; identity confidence reported, and an identity that cannot be re-established marked uncertain rather than silently matched
US3: a fingerprint for plan caching	P3	FR-008, FR-009 (2)	A per-frame fingerprint stable under sub-threshold noise, and comparison against a stored fingerprint reporting equal or different

Counts by category: 16 functional requirements, of which 12 are priority 1, 2 are priority 2 and 2 are priority 3; 11 non-functional requirements; 6 success criteria. Each functional requirement names the observable behaviour and the input that produces it, and none of them names a data structure, a package or an algorithm, which is what keeps the agents from inventing an implementation inside a requirement.

The requirements that carry the most weight are the ones that constrain failure rather than success: FR-005 (no regions when the difference is below the floor), FR-007 (an identity that cannot be re-established is marked, not guessed), FR-015 (fail explicitly rather than emit a partial document) and FR-016 (no network, no writes outside the declared path). They are what make the engine usable as a stage, and each has a boundary case in the test suite rather than only a happy path.

2.3 Non-Functional Requirements

Each non-functional requirement has a number, a target and a method, and the method is recorded in `docs/vv/plan.md` before the measurement is taken, so the target cannot be adjusted afterwards to fit the result.

ID	Attribute	Target	How measured	State
NFR-001	Performance	p95 at or below 12 ms, p99 at or below 25 ms per 1080p frame pair on one CPU core	make perf, per-pair timing, percentiles, GOMAXPROCS=1	met: p95 9.65 ms, p99 10.37 ms
NFR-002	Throughput	At least 30 frame pairs per second at 1080p on one core	make perf, sustained over 300 pairs	met: 78.2 per second
NFR-003	Memory	At most 128 MB over 10,000 frames, no growth with stream length	make memcheck, resident and heap sampled every 1,000 frames	met: 22.5 MiB peak resident
NFR-004	Determinism	Byte-identical output, independent of host, thread count and scheduling	Two runs encoded and compared; GOMAXPROCS 1 and 4	met in part: identical across thread counts on this host; cross-host is untested
NFR-005	Reliability	Zero false removals on the noise corpus	make accuracy, noise case, every pair	met: zero
NFR-006	Accuracy	Region F1 at or above 0.98 on 5,000 or more generated pairs	make accuracy, corpus with generated ground truth	met: 5,134 frame pairs, F1 1.0000
NFR-007	Usability	A new user produces a delta from their own screenshots within two minutes, from the README alone	Task timing with an external tester	not yet measured; the acceptance script exists and is not run
NFR-008	Portability	CPU only, no GPU, no network at runtime, Linux and Windows	Cross-build in CI, no-network test	partly met: cross-build done, no-network test outstanding
NFR-009	Maintainability	At least 80 percent line coverage on the geometry and identity modules	go test -cover	met: diff 81.9, identity 85.2
NFR-010	Compatibility	Schema changes versioned, and a consumer can reject a version it does not understand	Decode-time rejection test, exit code 3 from the command line	met

ID	Attribute	Target	How measured	State
NFR-011	Security	No network socket, no process execution, reads only the frames and configuration given, writes only to the declared output path	Two static checks over the syntax trees of every non-test file, and one dynamic check comparing the filesystem before and after a run	met: the checks fail when an import of <code>net/http</code> and a stray <code>os.WriteFile</code> are added, and pass again when they are removed

Two of the targets were met only after the first measurement failed, and the failures are recorded rather than replaced: latency measured 25.26 ms p95 before three optimisations (AUD-008), and the memory measurement passed while measuring nothing until it was rebuilt (AUD-007). The second case is the reason section 5.3 records what was wrong with the measurement and not only what the engine scored.

2.4 Technology Stack

Layer	Choice	Reason, and what was rejected
Language	Go 1.24 (toolchain 1.26)	Fast builds and tests, a standard library that decodes PNG without a dependency, and cross-compilation that makes the Windows requirement cheap. Rust was rejected for compile time against a two-day budget, C++ for build complexity, Python for the performance requirement
Images	Standard library only	One dependency fewer, and the formats in scope are PNG and raw RGBA, which the standard library covers
Contracts	JSON Schema (draft 2020-12) plus a hand-written Go validator	The schema is the consumer-facing contract and the validator enforces it in the engine, so a document that encodes is a document the schema accepts
Specification	GitHub Spec Kit v1.0.12	The course names it, it is command-driven, and its templates are overridable
Authoring agent	DeepSeek-backed agent harness (dsh)	Long-context authoring with repository reach, and the same repository reachable from three command surfaces
Review agent	Codex-based agent at low reasoning effort, in a separate context	Independence is the point: it sees the diff and the specification and is briefed to falsify rather than to agree
Hosting	GitLab at <code>gitlab.abo.fi</code>	Course requirement. The repository is local until the owner pushes
CI	GitLab CI, verify and test stages	Requirement traceability and the Go test suite run on every push; the jobs are checked locally before they are committed

The agent surfaces are the reason the stack section belongs in a report about specification: the same specification is reachable as Copilot prompts in `.github/prompts/`, `opencode` commands in `.opencode/commands/` and `dsh` skills in `.dsh/skills/`, so an agent can be changed without changing the process. Section 4.2 says what each surface is for.

2.5 Interface and API Specifications

The engine has three interfaces, each with a contract artefact in `specs/001-frame-delta-engine/contracts/`:

Interface	Contract	What it fixes
The delta document	<code>delta-document.schema.json</code>	Field names, types, required fields, the class-specific rules for <code>previousBounds</code> , the ordering of regions and conditions, and the identity fields FR-007 requires
The configuration document	<code>config.schema.json</code>	Every key, its range, its default and its meaning. An unknown key is an error with the offending key named, because a silently ignored setting is a wrong result that looks right
The library and command line	<code>interfaces.md</code>	The two Go interfaces, the exit codes (0 success, 1 usage, 2 input, 3 contract), what each subcommand can classify, and the consumer-facing rules for identity, covers and returns

The document is the important one, because it is what a later pipeline stage reads. It declares a schema version, carries no timing fields (so two runs are byte-identical, which is NFR-004), and states its ordering as part of the contract rather than as an implementation detail: regions sorted by top edge, then left edge, then identifier, then size, so a total order exists even for two regions that share a position and an identity.

2.6 How These Requirements and Specifications Were Created

The order was: constitution, then feature specification, then clarification, then plan, then contracts, then tasks, then implementation, with `analyze` run between plan and tasks. The commands and their outputs are recorded in `docs/analysis/` and the sessions in `docs/prompt-log/`.

What the agents produced and what was rejected:

- `/speckit.specify` produced the three user stories and the requirement skeleton in one pass. It was rewritten by hand in two respects: the success criteria were made measurable (the first version said "fast", the second said "p95 at or below 12 ms per 1080p frame pair on one core"), and an invented requirement about exporting to a database was deleted as out of scope.

- `/speckit.clarify` was the most valuable single command, because it asked the questions the first specification had left open. It produced, among others, the decision that an empty region-of-interest list means report nothing while an absent one means unrestricted, and that a viewport change is a condition rather than content.
- `/speckit.analyze` found two inconsistencies: a requirement traced to a task that did not exist, and a success criterion with no measurement. Both are fixed, and the reports are kept in `docs/analysis/`.
- The contract schema was written by an agent and then corrected by hand in three places the validator and the schema disagreed, which is exactly the class of defect the traceability check cannot see and the integration test can.

The rule applied throughout is the one in `AGENTS.md`: do not write implementation code that no task covers, and do not invent requirements. Where the specification turned out to be wrong about what the engine can know, the specification was changed in its own commit with the reason: the second acceptance scenario of user story 2 originally asked for an element hidden behind an overlay to keep its identity, which a pixel-only stage cannot do, and it now states the two events the engine can distinguish. That change is commit 17900ed, and `docs/prompt-log/0007-measurement-review.md` records why the first implementation could not satisfy the original wording.

3.1 Prompting Framework

The framework is a mix, chosen per activity rather than per project:

Activity	Technique	Why
Exploration before the specification	Role prompting with a named constraint ("you are choosing a system that a single agent team can finish and that a course can grade on process")	The first attempts failed by exploring systems with no constraint to optimise for, which section 1.2 records
Writing the specification	Structured, one Spec Kit command per step, with the command's own template as the prompt	The template asks for the fields the process needs; a free-form prompt produced a specification with no measurable criteria
Implementation	Chain of thought inside a task-bounded brief: goal, files, constraints, existing patterns, how to verify	An agent given a task from tasks.md has a bounded problem, and the brief carries the conventions it would otherwise guess
Verification	Adversarial role prompting: "your job is to falsify the claims below, not to comment on style"	A review prompt that asks for correctness returns code style; the one that asks for a counterexample returns defects
Measurement	No prompting: the harness is code, and the agent's role is to build and then try to defeat it	The most valuable findings in this project came from an agent trying to pass a measurement with a deliberately wrong implementation

The single most important choice is the last one. Three of the five review rounds found defects in the measurement rather than in the engine: a success criterion that could be satisfied by an engine with broken classification, a memory guard that passed with a deliberate per-frame leak, and a latency run inside the full test suite whose tail belonged to the machine rather than to the engine. Each is recorded as a defect with the fix, because a measurement that reports success while measuring nothing is worse than one that fails.

3.2 Master Prompts and System Prompts

The agent surfaces are configured in the repository rather than in a chat history, so the prompts are reviewable artifacts. The three that matter:

The repository constitution (.specify/memory/constitution.md) is the system prompt for every authoring session. Its operative rules, quoted from the file: requirements are written before code; requirement identifiers are stable; a deviation is recorded rather than edited away; every emitted document declares its schema version; the engine fails explicitly rather than emitting a partial document; and the engine never reaches the network. It is amended only with the reason recorded, and it has been amended once, for the identity fields FR-007 requires.

The authoring brief (AGENTS.md) is the working agreement. It fixes the order of operations (read the constitution, then the specification, then the plan and tasks, then the traceability matrix), the hard rules (no code without a task, update the

matrix in the same change, log the session, run `make check` before proposing a commit), and the autonomy boundary (local and reversible work needs no permission; anything that publishes or changes shared state does). It exists because an agent that has to be told the conventions each session will eventually forget one.

The review brief is not stored as a file but as a fixed shape, because it is sent fresh each round: name the repository and the commit range, state that the pass is read-only, list the claims to falsify numbered one by one, require a demonstrating case for each finding with a severity and a file and line, and ask for a verdict on whether the acceptance claim can be made. The prompt that produced round five is quoted in `docs/prompt-log/0007-measurement-review.md`, along with the two earlier rounds' briefs.

The expected result was that independent review would find defects the authoring pass could not. The actual result was stronger and less comfortable: it found that the *evidence* was not what it claimed. The later revisions to the brief ask about the measurement first and the code second.

3.3 Prompt Iteration Log

The full table is `docs/prompt-log/iteration-log.md`. One case, in the form the course asks for:

Original prompt	Failure it caused	Corrected prompt	Result	Verified by
"Build the accuracy and memory harnesses, run them, and record the numbers as evidence"	The harness passed while measuring almost nothing: 27 scored pairs instead of the 5,000 the criterion names, no classification checked, an answer key that contradicted the requirement, and a memory guard that a 512 byte per frame leak survived	"Score every adjacent pair, derive the answer key from the rendered pixels with an oracle independent of the engine, assert the classes each case states, require the sample size the specification names, and measure resident memory with a bound a half kilobyte per frame leak cannot survive. Then try to pass it with a deliberately wrong implementation before believing it."	5,134 frame pairs scored at F1 1.0000 with every asserted class correct; 22.5 MiB peak resident over 10,000 frames; the deliberate leak now fails both memory tests	make accuracy, make memcheck, the leak introduced and reverted, and the round recorded as AUD-007 in <code>docs/vv/results.md</code>

The general lesson, which the corrected prompt states as a rule: a measurement is not finished until an attempt to pass it with a deliberately wrong implementation has failed. Three of the 32 recorded defects were found by exactly that, and none of them by reading the code.

4.1 SDD Tool Kit

GitHub Spec Kit v1.0.12, pinned in the repository so the workflow cannot drift under it. The decision record is docs/adr/0001-adopt-spec-kit.md. It was chosen over three alternatives: a bespoke markdown process, which would have had no templates and no integration; a document-driven generator, which would have kept the specification outside the repository; and writing the process as prompts only, which is what parts of this project do in addition rather than instead.

What the kit contributes, and what was overridden:

Spec Kit artifact	Used for	Overridden
.specify/memory/constitution.md	The project rules every session reads	Amended once, with the reason recorded
.specify/templates/	The specification, plan, tasks and checklist shapes	Partly: the report skeleton and the V&V plan are repository documents, and the traceability matrix is generated by a script rather than a template
/speckit.specify, clarify, plan, checklist, tasks, analyze, implement	The workflow, one command per step	analyze output is captured to docs/analysis/ because the command is read-only by design
.specify/scripts/bash/	Creating the feature directory, resolving templates	Used unchanged

4.2 Development Environment

One Linux workstation (Ubuntu, Go 1.26 toolchain, 10 cores) with the repository reachable from three agent surfaces, all three configured in this repository:

- **GitHub Copilot prompts** in .github/prompts/, for editing inside an editor;
- **opencode commands** in .opencode/commands/, for a terminal agent with the repository in reach;
- **dsh skills** in .dsh/skills/, the harness used for the authoring rounds in this project.

A solo project still needs all three for one reason that is worth stating: the authoring pass and the review pass must run in *different* contexts, or the review is the author agreeing with themselves. Changing surface is the cheapest way to guarantee that, and it is what kept the five review rounds from inheriting the assumptions of the code they were reading. The review rounds in this project ran in a separate harness with no access to the authoring conversation, given only the repository, the commit range and the claims to falsify.

The models are named in the prompt log per session: a DeepSeek-backed agent for authoring, and a Codex-based agent at low reasoning effort for review. The reasoning effort is deliberate: the review task is to construct counterexamples against a

specification, which is bounded work, and the value comes from the questions asked rather than from the model's depth.

4.3 Deriving the Requirements Using the IDE and Coding Agents

The functional requirements came from the user stories by asking, for each story, what a consumer would have to be told and what the engine could be wrong about. That second question is where most of the interesting requirements come from: FR-007 (an identity that cannot be re-established is marked, not guessed), FR-015 (fail explicitly without a partial document) and NFR-005 (zero false removals on noise) are all answers to "how would this fail silently?".

The prompts that produced them, and what was rejected:

- "Write the functional requirements for a frame delta engine as observable behaviours, not as implementation" produced the skeleton in one pass. The agent's first version included "use a tile grid for comparison", which is a design decision that would have made the later measured removal of the tile grid a requirement change. It was rewritten as "report changed regions with bounds and magnitude".
- "What can this engine know for certain, and what can it only infer?" produced the distinction the whole design rests on: bounds and classes are observations, identity is an inference, and the uncertainty marker exists because the second is not the first.
- "List the ways a document consumer could be misled" produced the multiplicity rule (one element may be two regions), the ordering guarantee, and the rule that no timing field appears in a document.

The non-functional requirements were harder and were derived by asking what would be measured and by what command, before the command existed. That order matters: three of the ten targets failed their first measurement, and because the method was fixed in docs/vv/plan.md first, the response was to fix the engine rather than to adjust the target.

4.4 SDD Workflow and Implementation

The workflow as actually followed, with the gate that ends each step:

Step	Command	Output	Gate
1	/speckit.constitution	.specify/memory/constitution.md	The rules are quoted in AGENTS.md and enforced by review
2	/speckit.specify	specs/001-frame-delta-engine/spec.md	Every success criterion measurable, checked by hand
3	/speckit.clarify	The requirements notes and research decisions	Open questions in the specification brief are answered or recorded as research items
4	/speckit.plan	plan.md, research.md, data-model.md, contracts/	The contracts exist before the code they constrain
5	/speckit.checklist	checklists/requirements.md	Reviewer-owned items are never ticked by the author
6	/speckit.analyze	docs/analysis/analyze-2026-09-26.md	Five findings, all resolved or recorded
7	/speckit.tasks	tasks.md, 53 tasks	Every requirement has at least one task; checked by scripts/check_traceability.py
8	/speckit.implement, task by task	Code, tests, evidence	make check (traceability, format, vet, tests) before any commit; an independent review before a slice is called done
9	/speckit.converge (not yet used)	A gap list	Planned for the endgame, once the acceptance testing has run

Three deviations from the comfortable path are worth recording. First, the implementation started before the whole task list was written: the frame, error and document types existed while tasks 20 onwards were still being refined, because the interfaces they define had to be real before the plan could say anything true about them. The task list was updated in the same round. Second, the review gate was added after the fact: eleven code commits went in without an independent review, which the audit log records as a process failure, and the standing rule since is that a slice is not finished until a review in a separate context has tried to break it. That rule has since found 46 findings across six rounds.

4.5 Prompts

The prompts used during the specification and implementation rounds are in docs/prompt-log/, one entry per session, each with the verbatim prompt, the intent, what happened, what was corrected in the specification or the documents, and what the

agent got wrong. Seven entries exist. The ones that carry the most information for a reader are:

- 0005-system-selection.md: the selection round, including the four systems explored and rejected first;
- 0006-implementation-rounds.md: the first implementation round, including the missing review step;
- 0007-measurement-review.md: the round that falsified its own measurement, and the prompt change it caused;
- 0008-verification-tail.md: the end to end tests, the independent consumer, the demonstration on real rendered pixels, and the re-review that falsified five of the previous round's claims.

5.1 V&V Strategy

The strategy has five levels, and the reason for each is that the level below it cannot see the defect it catches.

Level	What it establishes	Mechanism	Where the result is recorded
Unit and package tests	That each rule behaves as its comment says, including the boundary cases	<code>go test ./...</code> , one test per decision named for the behaviour	Test names in docs/traceability.md
Integration and functional tests	That the parts agree: the engine, the differ, the identity map and the document validator	The corpus harness runs the real engine over generated frames and scores the documents	docs/vv/results.md
Measurement	That the non-functional targets hold, and that the harness that says so is not lying	<code>make accuracy</code> , <code>make perf</code> , <code>make memcheck</code> , each defeated deliberately before it is believed	docs/vv/evidence/
Requirement verification	That every requirement has a task, a test and recorded output, and that nothing is claimed on a file that does not exist	<code>make check-strict</code> , which runs the traceability script over the matrix and the specification	docs/traceability.md
User acceptance	That a person who has not read the code can use the engine from the README alone	The script in docs/vv/acceptance.md, run by external testers	Not yet run; the criterion is unverified and marked so

Threats to validity, stated because the numbers are only as good as their scope:

1. **The corpus is synthesised.** Ground truth is exact because the generator drew the frames, and the frames are flat panels with crisp edges, not screenshots of real applications. Real screens have antialiasing, subpixel text rendering and gradients, which the noise floor and the appearance signature were not tuned against. The corpus proves the rules, not the robustness.
2. **The accuracy metric is intersection over union at 0.5 against changed pixels.** An engine that reports slightly inflated boxes still scores, and one that reports very small regions does not. The score measures localisation rather than usefulness to a planner.
3. **Latency was measured on one machine**, an Intel i7-8700T virtual machine, one core, with the harness running alone. The tail on a loaded machine or a different microarchitecture is unknown, and `perf` refuses to report a contended run rather than reporting it as the engine's.
4. **The requirement is one number for a cost that follows the size of the change.** The asserted profile changes two elements per frame; a change covering half the screen costs about 40 ms and is reported separately. No single figure describes both.
5. **Determinism is demonstrated across thread counts on one host**, not across hosts or Go versions.
6. **The independent review is an agent, not a person.** It is a different context, a different model and a brief that requires counterexamples, which is a real substitute for a second pair of eyes and not the same thing. Section 6.1 says what the single-author history cannot show.
7. **The measurement can be gamed, and was.** Three defects were found in the harness rather than in the engine, all of the same kind: a green result that measured less than it claimed. Every measurement in this report has since been defeated deliberately at least once.

5.2 Code Audit Log

The classified log is `docs/vv/plan.md`, with one entry per finding, the class, the resolution and the test or measurement that verifies it. What it shows in summary:

Class	Count	What the class contained
Logical	27	Wrong region attribution and identity bugs: a translation reported as one box, a cover inheriting the covered element's identity, a partial change shrinking an element's footprint, identifiers reissued at a viewport change, an element that sat still being retired as occluded
Compliance	7	Cases where the evidence claimed more than the code did: validation permitting output the published schema rejects, a duplicate configuration member silently ignored, three cases where a measurement target reported success while measuring nothing or while being defeatable by a deliberately wrong implementation, a corpus generator that made every accuracy number irreproducible, a sample size overstated five times over, and a suite that failed while being read as a pass
Maintainability	1	Two CI jobs that could not pass, a memory guard that passed with no test, and a requirement table that grew without bound
Security	0	Stated as a result rather than an omission. The engine reads local files, writes only to a path it is given and never opens a socket; the nearest item is a decoder that accepts documents missing required fields, classified as compliance because the schema is the contract being broken

The pattern worth naming is that the compliance class is the one the process had to grow a defence for, and the defence is not one rule but three. A measurement is not finished until an attempt to pass it with a deliberately wrong implementation has failed, which is how four of the 35 defects were found. The number a measurement prints is not evidence until the code that prints it has been read, which is how the sample size turned out to be five times too large. And the exit code is the result, not the output, which is how a failing suite was read as a pass in the last round of this project.

The three defects in that last family all came from running the final verification carefully, and each had survived every earlier round because the number it produced was, in isolation, believable: F1 1.0000 is a perfect score whether the material is different on every run or not.

5.3 Test Execution Results

The running record is docs/vv/results.md, with raw output under docs/vv/evidence/. Fourteen rows are recorded, five of them withdrawn and rewritten in place after a review showed the first version was not trustworthy, with the defect that caused each withdrawal in the same table.

Current state, from the commands named:

```
| Measurement | Command | Expected | Actual | Status | Requirement | |---|---|---|---| |
Accuracy over generated frames | make accuracy | F1 at or above 0.98, zero false
removals, at least 5,000 pairs | 5,134 pairs and 27,094 regions, F1 1.0000, zero false
removals | pass | NFR-005, NFR-006, SC-001 | | Latency, one core, 1080p | make perf |
```

p95 at or below 12 ms, p99 at or below 25 ms | p50 8.44 ms, p95 9.65 ms, p99 10.37 ms | pass | NFR-001, SC-002 | | Throughput, one core | make perf | at least 30 frame pairs per second | 79.2 frame pairs per second | pass | NFR-002 | | Memory over 10,000 frames | make memcheck | at most 128 MB, no growth with stream length | peak heap 24.0 MiB, peak resident 22.2 MiB, identical heap at frames 1,000 and 10,000 | pass | NFR-003, SC-003 | | Allocation steady state | the allocation guard | no growth between two consecutive windows | the two windows agree | pass | NFR-003 | | Coverage | go test -cover | at least 80 percent on the geometry and identity modules | diff 81.9, identity 85.2 percent, and 87.2 and 96.4 on the other two | pass | NFR-009 | | Determinism | four runs encoded and compared | byte-identical output | identical across two thread counts and two collector settings | pass in part | NFR-004 | | Requirement traceability | make check-strict | every requirement traced to a task, a test and existing evidence | 33 rows, 29 verified, 0 errors, 0 warnings | pass | the process gate |

29 of 33 requirements are verified, meaning a test passes and its output is committed. The rest are in progress and the matrix says which, and the honest summary of the gap is this:

- **The identity requirements are verified against a specification that states their boundaries.** FR-006 and FR-007 were held back while a review's counterexamples stood; the two that remain are inherent to a single frame pair, they are written into the specification with their reason, and both are pinned by tests. Verifying them is a decision about what the requirement says, and the evidence file says so rather than implying the engine got better.
- **Two requirements are one step from verified and are held back deliberately:** NFR-004, because two of its three named conditions are demonstrated (thread count and collector scheduling, four configurations) and another host is not testable from here, and NFR-008, because the cross build is checked and running the suite on Windows is not.
- **Two requirements need testing that exists but has not been written:** FR-016 (no network, no stray writes) and NFR-008 (the Windows cross-build is in CI, the platform smoke test is not).
- **Two need a human who is not the author:** NFR-007 and SC-005 (usability), which is what the acceptance script is for, and SC-006 (an independent consumer), which needs a second implementation written against the schema alone.
- **One is a demonstration rather than a measurement:** SC-004 (sustained decision rate), which the demo pipeline is intended to produce.

One defect is documented as inherent rather than fixed: an element that disappears from exactly the footprint it occupied looks identical to a content change, and the engine reports a change, because the evidence that would justify a removal also fires

on a subtle repaint whose fill resembles its surroundings and a false removal is what the noise corpus exists to catch. Every other defect found in this project is fixed, including the decoder that accepted a document missing the required identity fields: presence is now checked against the raw JSON before the struct is built.

6.1 Version Control and Prompt Integration

Git activity carries 15 of the 50 course points, so the history is a deliverable rather than a by-product, and it is kept to one logical change per commit by design, as CONTRIBUTING.md states in its granularity section.

The numbers, all reproducible from the repository:

Measure	Value	How to reproduce
Commits	above 100, and the command gives the exact count	<code>git rev-list --count HEAD</code>
By type	27 docs, 23 feat, 15 fix, 6 plan, 3 test, 2 spec, 2 refactor, 1 perf, 1 chore (sums to 80)	<code>`git log --format='%s'</code>
Commits carrying a Spec: trailer	70	<code>`git log --grep='^Spec:' --oneline</code>
Carrying a Req: trailer	68	the same with <code>^Req:</code>
Carrying a Task: trailer	45	the same with <code>^Task:</code>
Carrying a Prompt: trailer	76	the same with <code>^Prompt:</code>
Days of work	2 (2026-09-26, 2026-09-27)	<code>`git log --format='%ad' --date=short</code>

The trailers are the link between a code change, the requirement it serves, the task that planned it and the session that prompted it, and they are the reason the traceability matrix can be checked by a script rather than by trust. `python3 scripts/check_traceability.py --require-specs` fails the build when a requirement has no row, a row names a task that does not exist, or a row claims verification without a test file and an evidence file that exist in the repository.

Issues and merge requests: the course asks for these to be visible, and this repository does not have them, because the work was done on one branch by one author with no remote until the owner pushes. That is a real gap against the course's expectation and it is stated rather than papered over with retrospective issues. What stands in its place is the review record: five independent review rounds, each with its findings, the fix and the test that proves the fix, recorded in `docs/vv/plan.md` (audit log) and `docs/prompt-log/`. The commit history is local and ready to push to `gitlab.abo.fi`, and the owner pushes it; the repository is not the agent's to publish.

6.2 Team Contribution Breakdown

The project is solo, so this section shows how the four roles were separated in time rather than between people, and what replaced the missing second and third reviewers.

Role	Who played it	How it was kept separate	Evidence
Specifier	Human, with an agent drafting	Wrote the constitution, the user stories and the success criteria before any code; corrected three requirements the agent had phrased as design	specs/001-frame-delta-engine/spec.md, commits beae933 and 17900ed
Prompt engineer	Human	Owned the briefs, changed them after each failure; the rule that a measurement must be defeated before it is believed came from a failure, not from a principle	docs/prompt-log/0007-measurement-review.md
Verifier	An agent in a separate context, directed by the human	Five rounds, each with a repository, a commit range and numbered claims to falsify; the authoring context was never shared with it	Findings in docs/vv/plan.md, docs/vv/results.md
Auditor	Human against CONTRIBUTING.md, plus the traceability check	The check runs in CI and in make check; the audit log records the one process failure (eleven commits without review) and the rule that followed	docs/vv/plan.md, CONTRIBUTING.md

Leverage, quantified rather than asserted: more than 100 commits in two days, 6,153 lines of Go against 5,813 lines of test and 4,661 lines of specification, plan and process documents, 53 planned tasks of which 52 are complete, 33 requirements of which 29 are verified, and 46 recorded findings of which 45 are fixed. The human wrote no implementation line by hand and read every one that was committed.

Where the human was the bottleneck is the honest part of this section, and there are three places:

1. **Deciding what the system should be.** Four candidate systems were explored in detail before the fifth was chosen, and two were rejected for reasons a single question about the course grading would have surfaced. No amount of agent throughput helps with a decision that has not been made.
2. **Accepting a measurement.** Every failing target and every falsified claim needed a human judgement about whether to fix the engine or change the claim, and the wrong answer is always available. The reviews found the defects; the human decided what to do about them.

3. **Specification changes and the acceptance gate.** The two specification changes (the identity fields FR-007 requires, and the restated acceptance scenario) and the review gate itself are serial: one decision at a time, each blocking the work that depends on it.

The claim this section could make, that one person directing agents produces the output of a much larger team, is supported by the commit and line counts. The claim it should not make is that the process was therefore fast: the specification gates, the review rounds and the acceptance testing stay serial and human, and they were the majority of the elapsed time.

7.1 Benefits of AI-Assisted SDD

What worked, with the measurement or the artifact that shows it:

- **The specification became the interface between the human and the agents.** 53 tasks, each with a file list, a constraint list and a verification method, meant an implementation session could be handed over and reviewed afterwards rather than dictated. Commits per session rose from a handful in the scaffold session to five to eight in the implementation rounds.
- **Tests written from the requirement rather than from the code.** Because the requirement text was the input, the corpus harness and the boundary tests exist at the level the requirement is stated: for example the noise floor has a test for a difference exactly at the floor and one level past it, which is a test nobody writes from reading the implementation.
- **Review in a separate context is cheap and effective.** Five rounds, 36 findings, no authoring context shared. The cost is minutes of wall clock per round, and the value is measured by what the findings would have cost later: the aliasing defect (AUD-001 in the first round) would have corrupted documents in a streaming consumer, and the tile-boundary defect (AUD-004) was invisible to every test that existed until the corpus reached 5,000 pairs.
- **Documentation kept pace with the code.** 4,661 lines of specification, research notes, plan and decision records against 6,153 lines of Go, because every non-obvious rule had to be written down to be implemented, and the writing was cheap once the decision was made.

How much time construction gained is hard to state honestly, so the report gives the count that can be verified instead: 6,153 lines of implementation and 5,899 lines of test in two working days, with a specification and a review trail that a reader can audit. The comparison that matters is not lines per hour but defects per requirement: 46 recorded findings across 33 requirements, of which 45 are fixed, is a rate that only holds because the review was as cheap as it was.

7.2 Core Bottlenecks and Challenges

Challenge	How it showed up	How it was overcome
Context limits	A review of the whole repository at once produced shallow findings; an implementation session that tried to hold the specification, the plan and the code drifted	Work sliced by task, reviews bounded to a commit range with numbered claims, and the durable state kept in the repository rather than in a conversation
Drift from the specification	Two requirements were phrased as design ("use a tile grid"), which would have made a later optimisation a requirement change	Requirements were rewritten as observable behaviour, and the traceability check keeps a requirement from existing without a task and a test
The evidence claiming more than the code did	Three defects in the measurement itself, each of the same kind: a green result that measured less than it claimed	Every measurement is now defeated deliberately before it is believed, and the rule is written into the prompt for each measurement round
A second memory of the same thing	The classifier kept its own copy of where the elements were, and the two copies disagreed	The identity map owns the geometry and the classifier reads it; the duplication was removed rather than reconciled
An undecidable question treated as decidable	A cover and a content change are the same rectangle, and the first implementation answered as if it knew	The question is now stated as undecidable in one frame pair (R17), answered with evidence rather than proof, and the residual boundaries are in the contract
Solo review blindness	The first eleven code commits went in with no independent review, and an author reviewing their own diff approves it	The review gate became a rule, and the authoring and review passes run in different contexts on different surfaces

The most expensive challenge was the third, not because it was the hardest technically but because it was the one that would have made the report wrong rather than the code wrong: a measurement that reports success while measuring nothing is worse than a failing one, because it is believed.

7.3 Lessons Learned and Best Practices

Each claim is attached to something in this project rather than to a general principle.

1. **Write the requirement so a test can fail it.** The success criteria that were rewritten from "fast" to "p95 at or below 12 ms per 1080p frame pair on one core" are the ones that eventually failed and were fixed; the vague ones would have passed by definition.
2. **State what the system cannot know.** The distinction between an observation (a region changed) and an inference (this is the same element) produced the

identity confidence and the uncertainty flag, which is the most defensible part of the contract. It also produced the honest no: a cover and a content change are the same rectangle, and the engine says so.

3. **Try to defeat your own measurement before believing it.** Three of the 32 recorded defects were found this way, and none by reading code. The version of this rule that generalises: for every green result, construct the wrong implementation that would also produce it, and check that it fails.
4. **One memory of one fact.** The classifier and the identity map both kept the element geometry, and the defect that survived longest came from the disagreement. Duplication in state costs more than duplication in code.
5. **Review in a different context, with a brief that demands a counterexample.** A review prompt that asks for correctness returns style; one that numbers the claims and requires a demonstrating case per finding returns defects. Five rounds, 36 findings, and the two most valuable were about the evidence rather than the code.
6. **Keep the specification in the repository, and let it be wrong sometimes.** Both specification changes in this project were corrections, made in their own commits with the reason, and the history of being wrong is more useful to a reader than a document that was always right because it was written last.
7. **Vibe coding and SDD are not a spectrum with a middle.** The unstructured phase was useful for exactly three bounded things and useless for the code. Where a deliverable is graded on process and verified against a contract, the specification is not overhead: it is the only thing that made the leverage legible, and the review rounds are what made it trustworthy.

7.4 Proposed Specification-Driven Development Workflow

A reusable workflow, derived from what worked here and from what did not. The diagram is `docs/process/sdd-workflow.svg`.

Phase	Objective	Activities	Inputs	Outputs	AI assistance	Human judgement
0. Frame the decision	Choose what to build, under the real constraints	Explore candidates against the constraints the owner actually has; write the decision record	The constraint list, the grading or the business rule	An ADR and a rejected-alternatives list	Drafting candidates and tradeoffs	The choice and name of the constraints before exploring
1. Constitution	Fix the rules every later session reads	Write the principles, the identifiers, the gate conditions	The ADR from phase 0	<code>.specify/</code> <code>memory/</code> <code>constitution.md</code>	Drafting, and checking the rules are testable	Which rules are not negotiable
2. Specify	State what the system must do, measurably	User stories in priority order, functional requirements as observable behaviour, non-functional requirements with a target and a method	The constitution	<code>spec.md</code>	Drafting, and generating the boundary cases	Rewriting requirements that name design choices, making every criterion measurable
3. Clarify	Answer the questions the specification left open	Run the clarification command; for each open question either decide or record it as research	<code>spec.md</code>	Requirements notes, research items	Asking the questions	Every answer is where design really decided
4. Contract	Fix the interfaces before the code	Data model, interface contract, schema, exit codes	<code>spec.md</code> and the answers	<code>contracts/</code> , <code>data-model.md</code>	Drafting the schema and the validator	The rules consumed, dependencies and the boundary conditions stated explicitly
5. Plan and analyse	Derive the work and check it for inconsistencies	Plan, then the analyzer; resolve or record every finding	The contract	<code>plan.md</code> , <code>tasks.md</code> , an analyze report	Generating the task list; the analyzer's cross-checks	Judging which findings matter, refusing to write code

Phase	Objective	Activities	Inputs	Outputs	AI assistance	Human judgement
						without task
6. Implement in slices	Build one slice at a time, verified	Task by task, tests from the requirement, one logical commit per change with the requirement and task trailers	tasks.md	Code, tests, commits	Writing the implementation and the tests	Review every decision when a slice is done
7. Verify by measurement	Turn every target into a number and every claim into evidence	Build the harness, run it, record raw output, defeat it deliberately	The non-functional requirements and docs/vv/plan.md	Evidence files, the results table	Building the harness	Fixing engine rather than the target setting sample
8. Review in a separate context	Falsify the slice before it is called done	Numbered claims, a read-only brief, a demonstrating case required per finding	The commit range and the specification	Findings with severity, and fixes with tests	The review pass itself, in a context that never saw the authoring	Deciding what to do and what to record in here
9. Accept	Check the system works for someone who did not build it	An acceptance script run by external testers, and a second consumer written against the schema alone	The README and the contract	Acceptance results, a consumer's issues	Drafting the script	Running with a user and believing result
10. Converge	Close the gap between the specification and what was built	Compare, list the gaps, decide for each whether to build it, change the requirement or record it	Everything above	The final report and the traceability matrix	The comparison	The final honest account

Two design choices in the workflow are deliberate and were paid for here. The review phase sits *between* implementation and done rather than at the end, because a defect found in a slice costs one round and the same defect found in the report costs a rewrite of the claim; and the measurement phase sits before the review, because the review's most valuable output in this project was about the measurement rather than the code.

Appendix: Evidence Index

Evidence	Location	What a reader can do with it
Specification, plan, research, data model, contracts, tasks	specs/001-frame-delta-engine/	Read the source of truth the agents were given, including the decisions and their reasons (R1 to R19 in research.md)
Requirement traceability matrix	docs/traceability.md	Check every requirement against its task, its test and its evidence; the three limits of the check are stated at the top of the file
Prompt records, per session	docs/prompt-log/	Read the verbatim prompts, what was expected, what happened, and what was corrected; iteration-log.md is the failure table
Analyzer reports	docs/analysis/	See the five inconsistencies the analyzer found and what happened to each
V&V plan, audit log, results, raw output	docs/vv/	Read the plan written before implementation, the classified audit log, the running results table, and the raw output of every measurement under evidence/
Decisions and deviations	docs/adr/	Read why Spec Kit, why a frame delta engine, and why Go
Process definition	.specify/memory/constitution.md, CONTRIBUTING.md, AGENTS.md	Read the rules the process ran under, including the commit granularity rule and the review policy
Workflow diagram	docs/process/sdd-workflow.svg	The feature workflow this repository committed to, referenced by section 7.4
Acceptance script	docs/vv/acceptance.md	The script for the external tester, which is written and not yet run

Reproducing every number in this report, in one block:

```
make check          # requirement traceability, and the process gate
make accuracy       # corpus scoring: precision, recall, F1, false removals
make perf           # latency percentiles and throughput on one core
make memcheck       # 10,000 frames at 1080p, resident and heap, allocation
                    steady state
```

```
go test -cover ./internal/diff/... ./internal/identity/... ./internal/  
fingerprint/... ./internal/stream/...  
git log --oneline --grep='Req:' # the code-to-requirement links
```

Every command in that block was run to produce the numbers above, and each of the measurements was also run against a deliberately wrong implementation to confirm it fails.